

System Design Write-Up: Society Maintenance Tracker

Name : Gudikandula Sai Charan Chary

Mail : gudikandula.sai2023@vitstudent.ac.in

Neopat ID : D4P1V0G2

[Live Project URL](#)

Admin Credentials

Username: saicharanb3h@gmail.com

Password: password123

Resident Credentials

Username: gudikandula.sai2023@vitstudent.ac.in

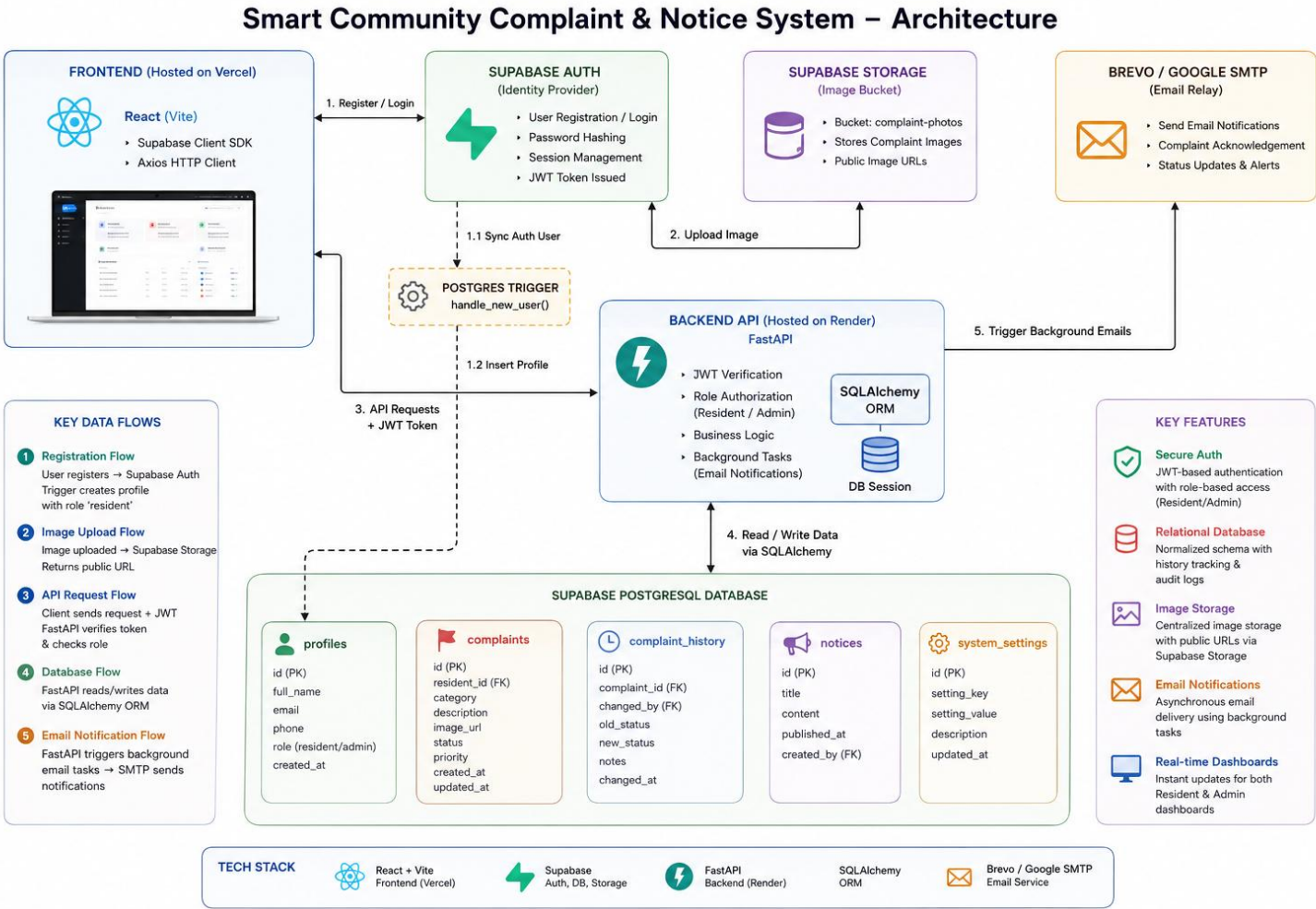
Password: qwertyuiop123

Contents

1.Architecture	2
2.Database Schema	3
3.Complaint History Audit Model	3
4. Dynamic Overdue Detection	4
5.Photo Handling & Storage Architecture	4
6. Asynchronous Notification Pipeline	5
7.REST API Design Overview.....	6
8. Api Endpoint Specifications	6

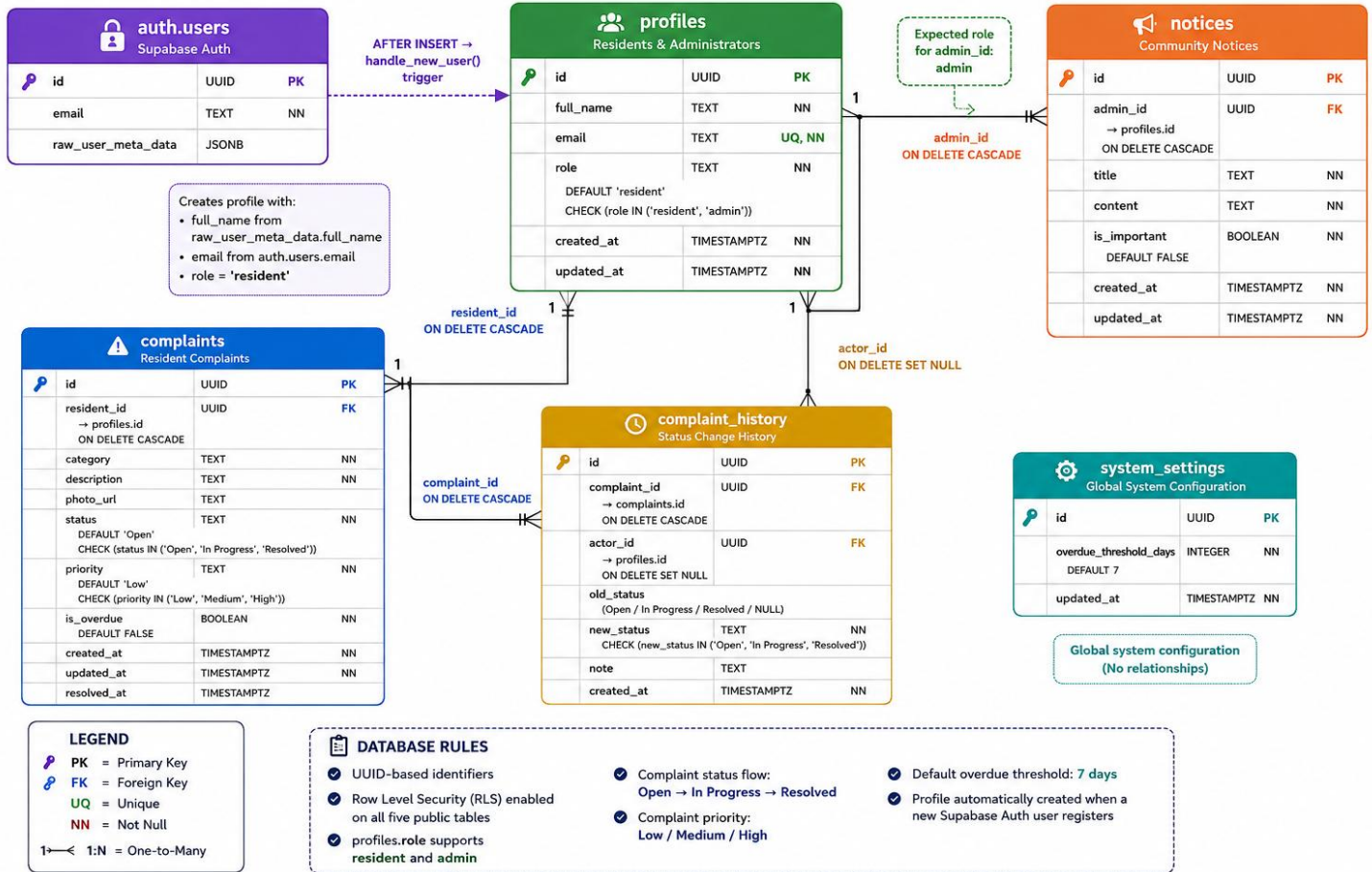
This document describes the engineering design and implementation details for the core services of the **Society Maintenance Tracker** application: The API design & endpoints, complaint history audit trail, dynamic overdue calculations, photo upload workflow, and background notification pipeline.

1.Architecture



2.Database Schema

Smart Community Complaint System – Database Schema



3.Complaint History Audit Model

The application enforces a strictly audited, state-validated lifecycle for maintenance complaints, transitioning from Open → In Progress → Resolved.

Schema & Auditing

Every status transition is logged in the complaint_history table (see databaseschema.md). This model tracks:

- **old_status / new_status:** The state transition boundaries.
- **actor_id:** A foreign key pointing to the user profile responsible for the change.
- **note:** Optional text containing diagnostic details, updates, or resolution reasons.

Transaction Safety & Lifecycle Rules

To prevent database inconsistency, all status updates and audit logs are wrapped inside database transactions using SQLAlchemy. The system implements the following state restrictions:

- **Closed State Locking:** Once a complaint reaches the Resolved state, its lifecycle is permanently locked. The system rejects any status updates to a resolved complaint to maintain audit integrity and prevent reopening closed issues.

- **Auto-Timestamping:** Transitioning to Resolved automatically updates the complaint's resolved_at column.

4. Dynamic Overdue Detection

Instead of using cron jobs to statically mark issues as overdue, the application uses a **dynamic, configuration-driven query approach** to flag overdue items.

Configuration

Administrators configure an age threshold parameter (overdue_threshold_days) saved in the system_settings table. It defaults to 7 days.

Computational Logic

Overdue status is calculated on-the-fly when complaints are queried:

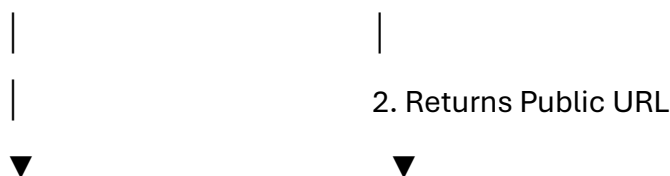
- **Criteria:** A complaint is overdue if:
Current Time – created_at > overdue_threshold_days **AND** the complaint's current status is **not** Resolved.
- **Exclusion of Closed Issues:** Closed issues are explicitly excluded from overdue calculations. If an issue was resolved after the threshold, it is not flagged as overdue.
- **Performance Optimization:** Because this logic is executed on read operations, indexing the created_at and status columns guarantees sub-millisecond query execution even under heavy database loads.

5. Photo Handling & Storage Architecture

The application handles media files securely, minimizing backend load and verifying file sizes.

text

[React Client] —1. Validate File Size/Type —> [Supabase Storage Bucket]



[FastAPI Backend] ◀—3. POST /complaints (includes image URL)

1. **Client-Side Verification:** Before uploading, the frontend validates the file format (JPEG, PNG, WEBP) and verifies that the file size is under 5MB.
2. **Direct Uploads:** The React client uploads the file directly to the Supabase complaint-photos storage bucket. This reduces backend resource consumption and avoids handling raw multipart file streams on Render.

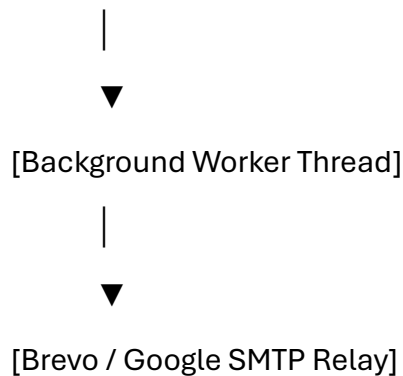
3. **Backend Validation:** When the client submits the complaint details, the backend performs a security validation on the provided image URL to ensure it points to the correct Supabase bucket domain and contains a valid image file type extension.
4. **Database Storage:** The public storage URL is saved in the complaints.photo_url column and rendered in the frontend detail views.

6. Asynchronous Notification Pipeline

To prevent blocking database transactions on third-party network requests, email notifications are processed asynchronously using **FastAPI BackgroundTasks**.

text

[HTTP Request] → [Update Database] → [Enqueue Background Task] → [Return HTTP 200 OK]



Process Flow

1. **Trigger Events:** Notifications are triggered by two actions:
 - When an admin updates a complaint's status.
 - When an admin publishes a notice marked as "important" (sending an announcement broadcast to all residents).
2. **Task Enqueuing:** Instead of sending the email synchronously during the HTTP request, the router registers a task with FastAPI's BackgroundTasks runner and returns an immediate 200 OK response to the client.
3. **SMTP Relay:** The background worker runs in a separate thread, connecting to the configured SMTP server (Brevo or Google SMTP) to transmit the HTML email.
4. **Fallback Mechanism:** If the SMTP server is down or rejects the credentials, the system catches the exception and logs the email content to stdout. This ensures that database writes are never rolled back due to email delivery failures.

7.REST API Design Overview

All endpoints are hosted under the base prefix `/api` and require a JWT token in the Authorization: Bearer <token> header (except pre-login flows handled directly by Supabase Auth). The API returns JSON payloads and utilizes standard HTTP status codes:

- 200 OK / 201 Created for successful requests.
- 400 Bad Request for validation failures (e.g. invalid file type or locked status transition).
- 401 Unauthorized / 403 Forbidden for missing tokens or insufficient role permissions.
- 404 Not Found for missing resources.

8. Api Endpoint Specifications

1. Authentication & Profile

Endpoints for fetching profile details of the authenticated session.

- **GET `/api/auth/me`** (Access: *Authenticated Users*)
 - **Description:** Retrieves the logged-in user's profile details.
 - **Response:** Profile object containing id, full_name, email, role, and registration timestamps.

2. Complaints Management

The core pipeline for filing and resolving issues.

- **POST `/api/complaints`** (Access: *Resident*)
 - **Description:** Registers a new complaint.
 - **Request (Multipart Form):** category (string), description (string), and optional photo (binary image).
- **GET `/api/complaints`** (Access: *All Authenticated*)
 - **Description:** Retrieves complaints. Residents only retrieve their own complaints; Admins receive all complaints in the society.
 - **Query Parameters:** page, limit, status, priority, category, search, and overdue (boolean).
- **GET `/api/complaints/{id}`** (Access: *All Authenticated*)
 - **Description:** Retrieves details for a specific complaint, including its complete chronological state timeline (history array).
- **PATCH `/api/complaints/{id}`** (Access: *Admin*)
 - **Description:** Modifies complaint state.

- **Body:** priority (optional string), status (optional string), and note (optional audit message).

3. Notice Board

Used by administrators to broadcast announcements.

- **GET /api/notices** (Access: *All Authenticated*)
 - **Description:** Lists announcements. **Important/pinned notices are automatically sorted to appear first.**
- **POST /api/notices** (Access: *Admin*)
 - **Description:** Creates an announcement. If `is_important` is true, it triggers a background email broadcast to all resident profiles.
- **PUT /api/notices/{id} & DELETE /api/notices/{id}** (Access: *Admin*)
 - **Description:** Modifies or removes notices.

4. Settings & Metrics

- **GET /api/settings** (Access: *All Authenticated*)
 - **Description:** Reads system configuration parameters (such as `overdue_threshold_days`).
- **PUT /api/settings** (Access: *Admin*)
 - **Description:** Updates the overdue threshold setting.
- **GET /api/dashboard/admin** (Access: *Admin*)
 - **Description:** Compiles dashboard analytics: total complaints count, active complaints, overdue count, status pie counts, category bar splits, and 30-day trends.
- **GET /api/dashboard/resident** (Access: *Resident*)
 - **Description:** Compiles resident-level summaries: total raised, resolved, in progress, and lists recent complaints.